

Amortized Analysis: Proving Append is $O(1)$

Programming and Data Structures — Week 2, Lecture 3

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to state precisely what the phrase “amortized $O(1)$ ” means, rather than repeating it as an unexamined slogan; prove, using the accounting method, that a sequence of n appends to a doubling dynamic array costs $O(n)$ total time; and explain why this result is entirely compatible with the fact that any single append can still cost $O(n)$ in the worst case, when it happens to trigger a resize.

Outline

Today covers the apparent contradiction between “append is $O(1)$ ” and “some appends visibly copy the whole array”; the accounting method, using the metaphor of prepaying rent; the full worked proof that n appends cost $O(n)$ total; the MTech-track rigorous version using a potential function; an in-class quiz; and a summary.

The tollbooth analogy

Before the formal argument, hold a physical analogy: a highway tollbooth charges every car exactly the same flat fee, even though the actual cost of maintaining the road varies considerably from day to day. On an ordinary day, maintenance is cheap and the flat fee collected is mostly banked as profit. On the day a pothole needs urgent, expensive repair, that repair is paid for out of the profit banked on all the preceding ordinary days — no individual driver is charged a special “pothole surcharge” that day. Amortized analysis applies exactly this reasoning to computation: charge a flat, slightly inflated cost against every operation, bank whatever is not immediately spent, and use that bank to pay for the rare expensive operation when it occurs. The rest of this lecture makes this rigorous for the specific case of a doubling dynamic array.

The apparent contradiction

Every textbook states that `list.append(x)` runs in $O(1)$ time. Lecture 2 demonstrated, by tracing capacity growth by hand, that some individual append calls trigger copying the entire array — an operation that is obviously $O(n)$ for that specific call, not $O(1)$. Both statements are true simultaneously, and resolving that apparent contradiction is this lecture's entire purpose.

The accounting method: an analogy first

Imagine that every single append costs exactly 3 coins, paid up front, regardless of whether that particular append is cheap or expensive. One coin pays for actually writing the new element into its slot. The other two coins go into a savings jar and are not spent yet. This “pay a flat amount every time, save the surplus” idea is the entire mechanism of the accounting method — the rest of this lecture is checking that the jar never runs empty.

Where the saved coins get spent

When a resize occurs — say, doubling from capacity k to capacity $2k$ — exactly k existing elements must be copied into the new array. Each of those k elements deposited 2 coins into the jar back when it was first inserted, and none of those coins have been spent since the previous resize. That gives $2k$ coins sitting in the jar, which is exactly enough to pay for copying all k elements (1 coin each) with k coins left over — enough to start the jar again for the next round of growth.

The proof, step by step

The full proof runs as follows. Every append deposits a flat 3 coins — 1 spent on the insertion itself, 2 saved. Over a sequence of n appends, the total coins spent is therefore never more than $3n$, by construction — this is true regardless of exactly when resizes happen. Step 2 above shows the jar balance never runs negative, meaning every resize is fully paid for by coins already banked, with none borrowed from the future. Since $3n$ coins cover the entire sequence of n appends, the total real cost of n appends is $O(n)$, which means the *average* cost per append — the amortized cost — is $O(n)/n = O(1)$.

What this does *not* say

Amortized $O(1)$ does not claim that every individual append costs $O(1)$ — the specific call that triggers a resize genuinely does copy k elements and genuinely does cost $O(k)$ for that one call. What the proof shows is that this occasional expensive call is exactly paid for by the savings accumulated during the many preceding cheap calls, so that averaged over any sufficiently long sequence, the cost per append

settles to $O(1)$. This is why “amortized” is not the same claim as “worst-case” — a single append in a real-time system with a hard per-call deadline could still miss it.

Full worked trace — the coin jar over 8 appends

Tracing the coin jar by hand over five appends confirms it never runs negative. Every append deposits 3 coins and spends 1 immediately on the insertion, banking 2. Append 2 triggers a resize copying 1 element, spending 2 coins from the jar — exactly what was banked. Append 3 triggers a resize copying 2 elements, spending 4 coins; the jar had 2 from before plus 2 newly banked this round, giving exactly 4 available, spent in full. Append 5 copies 4 elements, spending 8 coins, again exactly matched by what has accumulated. The jar balance column never goes negative at any point — this is the concrete, numeric version of the proof that follows.

The proof, step by step

$$\Phi(\text{array}) = 2 \cdot \text{length} - \text{capacity}$$

The rigorous version of this argument replaces the coin-jar metaphor with a potential function, $\Phi(\text{array}) = 2 \cdot \text{length} - \text{capacity}$, which plays exactly the role of the savings jar in numeric form. The amortized cost of any operation is defined as its actual cost plus the change in potential, $\Delta\Phi$, that the operation causes. Proving amortized $O(1)$ then reduces to showing Φ never goes negative and that the amortized cost, computed this way, is bounded by a constant for every operation — which is a mechanical calculation once Φ is chosen well.

MTech addition — checking the two cases

Checking the two possible cases confirms the claim. When no resize happens, the actual cost is 1 (a single write), length increases by 1 and capacity is unchanged, so $\Delta\Phi = 2$, giving amortized cost $1 + 2 = 3$. When a resize doubles the capacity from k to $2k$, the actual cost is $k + 1$ (k copies plus the new insertion), and $\Delta\Phi$ works out to $2 - k$, giving amortized cost $(k + 1) + (2 - k) = 3$. Both cases produce exactly the same constant amortized cost, 3, which is the formal proof that append is amortized $O(1)$ — the coin-jar argument earlier was this same calculation in plain language.

Full worked trace — the potential function over the same 5 appends

Reproducing the exact same 5-append sequence using the potential function $\Phi = 2 \cdot \text{length} - \text{capacity}$ gives the same conclusion numerically. After append 1, $\Phi = 2(1) - 1 = 1$, up from $\Phi_0 = 0$, so $\Delta\Phi = 1$ and amortized cost is $1 + 1 = 2$ (this row does not hit the steady-state constant of 3 because it is the very first operation, before the jar has settled). After append 3, which resizes from capacity 2

to 4 copying 2 elements: actual cost is $2 + 1 = 3$, Φ goes from 2 (after append 2) to $2(3) - 4 = 2$, so $\Delta\Phi = 0$, giving amortized cost $3 + 0 = 3$. After append 5, which resizes from capacity 4 to 8 copying 4 elements: actual cost is $4 + 1 = 5$, Φ goes from 4 (after append 4) to $2(5) - 8 = 2$, so $\Delta\Phi = -2$, giving amortized cost $5 + (-2) = 3$. Every steady-state row settles to the same constant, 3, matching the coin-jar trace exactly — two independent methods, one answer.

Why this matters beyond the exam

This result is not merely a classroom exercise. Every later week of this course treats append, push, and similar operations as “basically free,” and this lecture is what actually justifies that assumption rather than merely asserting it. In technical interviews, a question like “is this $O(1)$?” frequently hinges on exactly this distinction between amortized and worst-case cost, and correctly saying “amortized $O(1)$, but worst-case $O(n)$ for the resizing call” is the complete, correct answer rather than a pedantic detail. It is also worth knowing that some real systems — flight control software, medical devices with hard real-time deadlines — genuinely cannot tolerate an occasional $O(n)$ call even if the average is fine, which is why such systems sometimes use pre-allocated, fixed-size arrays deliberately, trading flexibility for a guaranteed worst case.

Quiz — the tollbooth, applied

Return to the tollbooth analogy: the toll is a flat 5 dollars per car, but this particular month’s road repair bill is far larger than 5 dollars times the number of cars that passed this month. Before checking the answer, decide: does this single expensive month mean the flat-fee pricing model has failed?

Quiz — the tollbooth, answer

No, not necessarily — provided the surplus banked during previous, cheaper months is enough to cover this month’s expensive repair. The amortized claim was never that every individual month costs exactly the average; it was that costs even out *over a sufficiently long run*. This is the same logic protecting the amortized-append claim: one particular append being expensive, because it happens to trigger a resize, does not contradict the claim that the $O(1)$ figure is correct *on average*, precisely because the preceding cheap appends banked enough surplus to cover it.

Exercise

A dynamic array’s `pop()` operation can also shrink its capacity once the array becomes very empty, to reclaim memory. Using the coin-jar reasoning from this lecture, sketch why a common rule — shrink the capacity by half only once the array falls to one quarter full, rather than one half full — avoids a pathological back-and-forth of repeated resizing when elements are appended and popped alternately near a boundary.

A second exercise — a stack of function calls

As a second exercise, consider a call stack — covered properly next week — that grows by one frame on every function call and shrinks by one frame on every return. If this stack were backed by a doubling dynamic array, could a deep chain of recursive calls followed by returning all the way back out ever trigger repeated, needless resizing? Think specifically about what happens to capacity on the way down (growth, handled by this lecture's proof) versus what should happen to capacity on the way back up (shrinking, which the earlier exercise on popping addressed) before next week's lecture covers stacks formally.

Quiz — amortized or worst-case?

For each of these two claims, decide whether it is a statement about amortized cost or worst-case cost, before checking the answer: “1,000 appends in a row take roughly 1,000 units of time, in total”; and “this specific append, right now, might take up to n units of time.”

Quiz — answers

Claim 1 is an amortized-cost statement: it describes the average behavior across a long sequence of operations, which is exactly what the coin-jar proof and the potential-function proof both establish. Claim 2 is a worst-case statement about one specific call in isolation, and it remains true even though claim 1 also holds — a resize-triggering append really can take $O(n)$ time on its own, it is simply rare enough, and paid for by preceding cheap calls, that the long-run average stays $O(1)$.

Summary

Amortized $O(1)$ does not mean every individual call runs in $O(1)$ time; it means the *average* cost across a long sequence of n calls is $O(1)$, even though individual calls vary. The accounting method formalizes this by having every operation overpay slightly on the cheap calls, banking the surplus to cover the occasional expensive one — and both the coin-jar argument and the more rigorous potential-function method prove the same result: n appends to a doubling dynamic array cost $O(n)$ total. Next lecture moves to stacks and queues, the first data structures in this course built as a thin, disciplined interface on top of the dynamic array just analyzed.